

Monitoramento de Quedas com IoT

Disciplina: Sistemas Distribuídos — Engenharia da Computação, UFMA

Alunos: Ana Iara Loayza Costa, Hígor Pinheiro Costa,

José Victor Brito Costa, Milena Freire Britto Neves

Repositório GitHub: <https://github.com/josevbrito/load-test-fall-detection-iot>

Repositório GitHub: <https://github.com/milenafbn/fall-detection-iot>

O projeto consiste no desenvolvimento de um sistema distribuído para detecção de quedas utilizando IoT. A arquitetura é composta por três camadas: borda, comunicação e *middleware*. Na camada de borda, um ESP32 conectado ao sensor MPU6050 coleta dados de aceleração e rotação e executa localmente o algoritmo de detecção de quedas. A comunicação com a nuvem é realizada via protocolo MQTT, utilizando o modelo *publish/subscribe* para transmissão assíncrona de telemetria e eventos. Na camada de *middleware*, o Things-Board recebe e armazena os dados, executa regras de negócio para geração de alarmes e disponibiliza *dashboards* para monitoramento em tempo real. A solução permite avaliar conceitos de sistemas distribuídos, mensageria e escalabilidade em ambientes IoT.

1. Arquitetura do Sistema

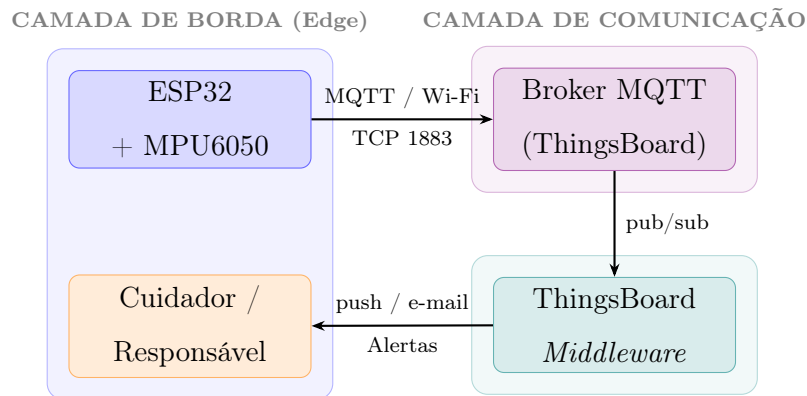


Figura 1: Arquitetura distribuída em três camadas do sistema de monitoramento

Camada de Borda: o ESP32 coleta dados inerciais via MPU6050 (I²C) e executa localmente um algoritmo de máquina de estados que detecta quedas pelo padrão: pico de aceleração acima de 2,0 g seguido de magnitude abaixo de 0,3 g por 1.500 ms, transmitindo apenas eventos confirmados.

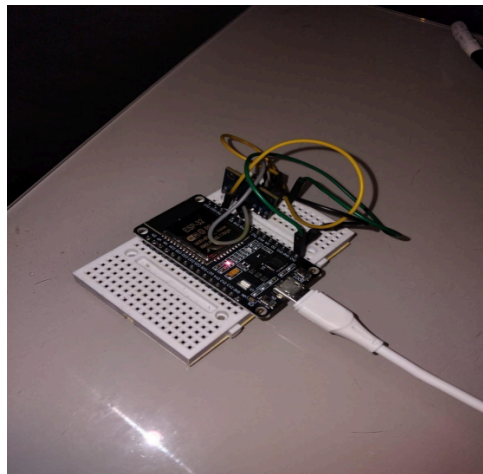


Figura 2: Dispositivo embarcado: ESP32 com sensor MPU6050 na camada de borda

Camada de Comunicação: a transmissão ocorre via Wi-Fi 2,4 GHz com protocolo MQTT (*publish/subscribe*), usando o tópico `v1/devices/me/telemetry` para telemetria contínua (1 s) e `v1/devices/me/attributes` para alertas de queda, roteados pelo *broker* do ThingsBoard na porta 1883. A Figura 3 exibe os valores dos eixos X, Y e Z chegando ao *dashboard* após a transmissão.

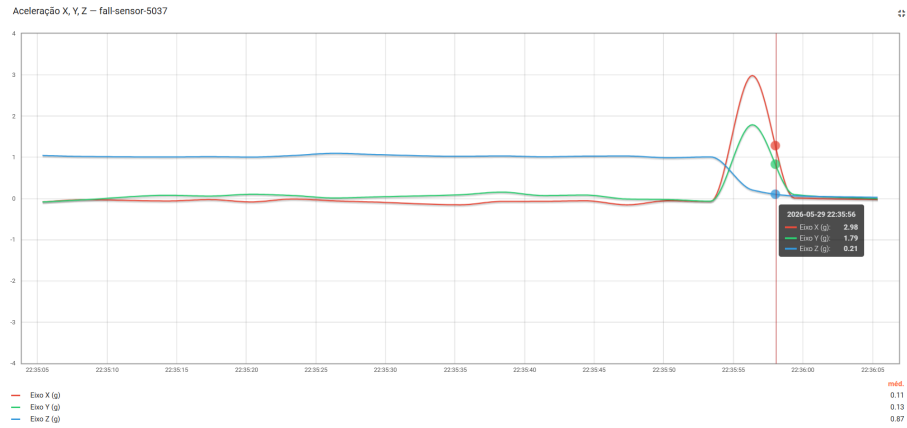


Figura 3: Telemetria de aceleração nos eixos X, Y e Z recebida pelo ThingsBoard

Camada de Nuvem: o ThingsBoard recebe os dados, armazena em série temporal, aplica regras automáticas via *Rule Engine* disparando alarme `QUEDA_DETECTADA` com severidade `CRITICAL` quando `fall_detected: true` e disponibiliza *dashboard* com magnitude e leituras por eixo acessíveis remotamente. A Figura 4 mostra o painel combinado de magnitude atual com o indicador de detecção de queda.



Figura 4: Painel ThingsBoard: magnitude atual e detecção de queda em tempo real

A Figura 5 exibe a série temporal de magnitude registrada durante um evento de queda: a curva sobe abruptamente no momento do impacto (pico acima de 2,0 g) e desce para próximo de zero durante o período de repouso subsequente, confirmando o padrão esperado pelo algoritmo de detecção.

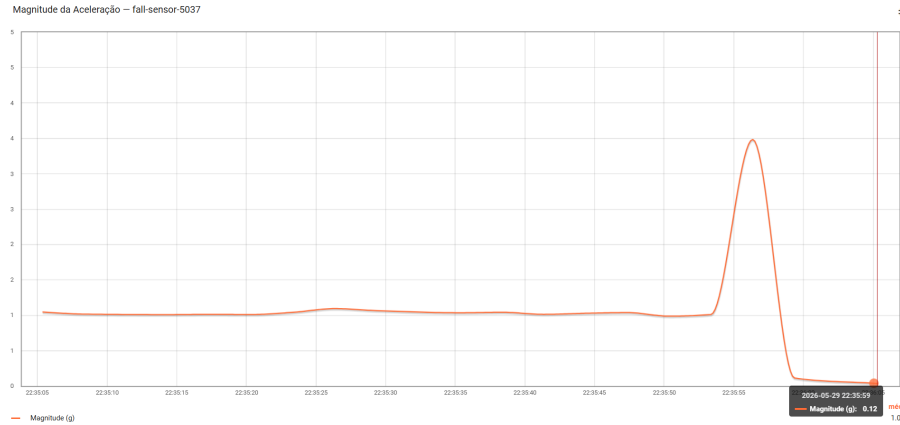


Figura 5: Série temporal de magnitude no ThingsBoard: elevação no impacto e queda no repouso pós-evento

2. Dificuldades e Problemas Durante a Execução

Ao longo da fase de execução do projeto, a equipe enfrentou desafios técnicos que exigiram investigação, adaptação e aprendizado prático. Os principais são descritos a seguir.

2.1 Configuração do Ambiente WSL para Upload de Firmware

O desenvolvimento do firmware foi realizado em ambiente Windows com WSL (*Windows Subsystem for Linux*). O processo de gravação do firmware no ESP32 via USB apresentou dificuldades de reconhecimento da porta serial pelo subsistema Linux, uma vez que dispositivos USB físicos não são nativamente acessíveis no WSL sem configurações adicionais. A solução envolveu o uso de ferramentas de redirecionamento de porta USB e ajustes nas permissões do ambiente.

2.2 Limitação de Conectividade Wi-Fi do ESP32

O microcontrolador ESP32 suporta exclusivamente redes Wi-Fi na faixa de 2,4 GHz, não sendo compatível com redes 5 GHz. Em ambientes onde o roteador disponível operava preferencialmente na faixa de 5 GHz ou em modo dual-band sem separação de SSIDs, o dispositivo não conseguia se conectar. Foi necessário identificar e configurar uma rede 2,4 GHz dedicada para os testes.

2.3 Calibração dos Limiares de Detecção

A definição dos limiares de aceleração para a detecção de queda exigiu um processo iterativo de calibração com o sensor físico. Os valores inicialmente planejados, baseados em

referências teóricas, geraram falsos positivos em movimentos bruscos que não caracterizavam quedas reais. A equipe realizou múltiplos testes controlados, ajustando o limiar de pico de aceleração para 2,0 g e o período de inatividade subsequente para pelo menos 1.500 ms com magnitude inferior a 0,3 g, combinação que apresentou melhor equilíbrio entre sensibilidade e especificidade nas condições testadas.

2.4 Lacunas (*Gaps*) na Telemetria

Durante os testes de desenvolvimento, foram observadas interrupções nas séries temporais exibidas no *dashboard* do ThingsBoard. Essas lacunas corresponderam aos períodos em que o dispositivo era desconectado durante o movimento.

2.5 Pendência: Sistema de Alertas

Ao momento da elaboração deste relatório, o sistema de alertas aguarda testes controlados de queda para atestar a confiabilidade dos limiares calibrados; a lógica no ThingsBoard já está operacional e dispara o alarme `QUEDA_DETECTADA` corretamente dentro da própria plataforma.

3. Execução do Load Test

Para avaliar a escalabilidade e os limites do *middleware* ThingsBoard CE sob carga realista de dispositivos IoT, foi desenvolvido um motor de teste de carga em Python assíncrono (`asyncio` + `aiomqtt`) capaz de simular simultaneamente milhares de dispositivos ESP32 publicando telemetria MQTT. Cada dispositivo virtual executa a mesma máquina de estados do firmware real, incluindo simulação de dados MPU6050 com ruído gaussiano, deriva GPS e eventos de queda com probabilidade configurável.

3.1 Metodologia

O load test foi executado com o seguinte cenário:

- **Baseline** — 1.000 dispositivos, 1.000 requisições cada, intervalo de 200 ms.

Em ambos os casos, o limite de conexões simultâneas foi fixado em 5.000 (semáforo *asyncio*), o modo de queda utilizado foi *session* (no máximo uma queda por dispositivo por sessão, com probabilidade de 5%) e o ThingsBoard foi executado localmente via Docker. O progresso foi acompanhado em tempo real por meio de um painel TUI (*Terminal UI*) baseado na biblioteca Rich, conforme ilustrado na Figura 6.

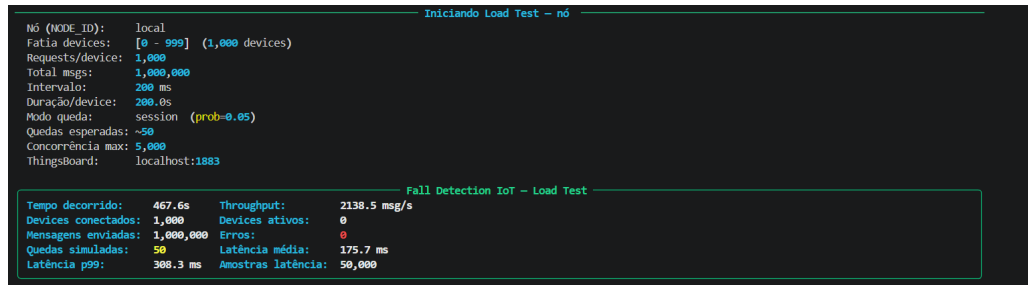


Figura 6: Painel TUI do load test: métricas em tempo real durante a execução

3.2 Monitoramento com Prometheus e Grafana

Para capturar métricas com granularidade temporal durante a execução, o motor de load test foi instrumentado com a biblioteca `prometheus_client`, expondo um *endpoint* HTTP `/metrics` na porta 8001 de cada nó. Um servidor Prometheus (raspagem a cada 5s) e um servidor Grafana foram adicionados à infraestrutura Docker, com provisionamento automático de *datasource* e *dashboard*. As Figuras 7 e 8 apresentam os painéis resultantes durante o cenário de escala.

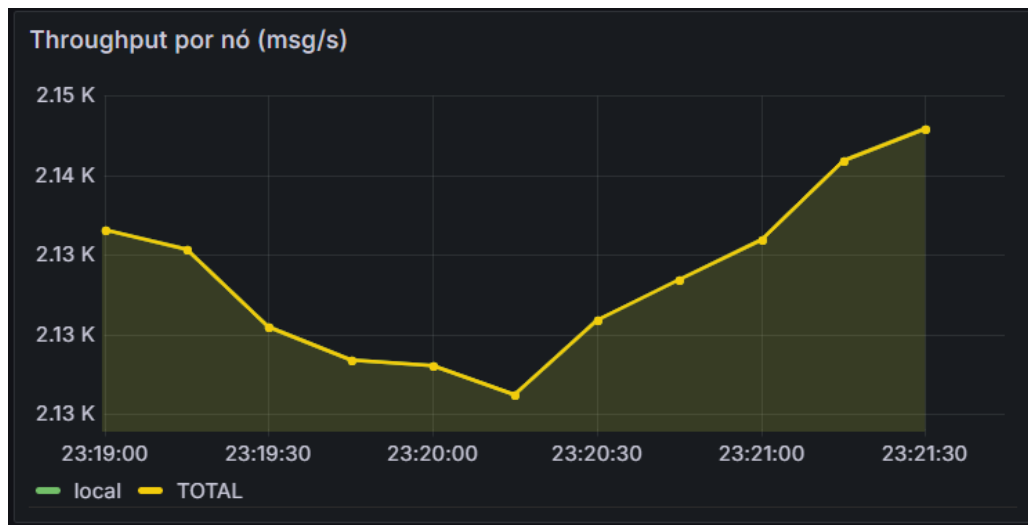


Figura 7: Grafana: throughput por nó sensor ao longo do tempo (msg/s)

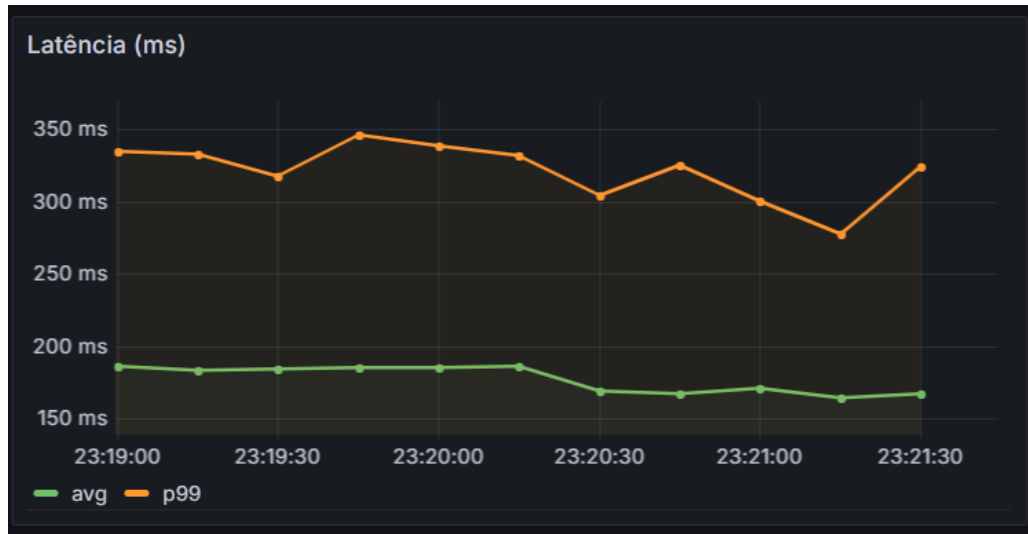


Figura 8: Grafana: latência média e p99 ao longo do tempo (ms)

3.3 Resultados Preliminares

A Tabela 1 consolida as métricas registradas ao final de cada cenário.

Tabela 1: Comparativo de métricas entre os cenários de load test

Métrica	Baseline (1k devices)
Dispositivos simulados	1.000
Requisições por dispositivo	1.000
Intervalo entre publicações	200 ms
Total de mensagens	1.000.000
Quedas simuladas	50
Throughput (msg/s)	2.137,5
Latência média	175,7 ms
Latência p99	308,3 ms
Tempo total de execução	467,8 s
Taxa de erros	0 %

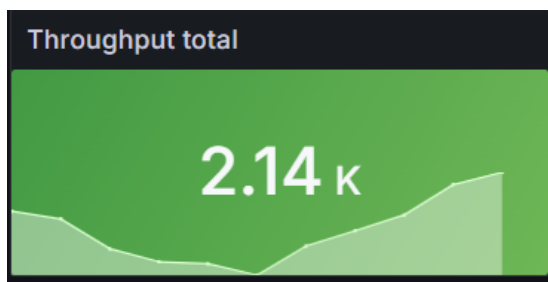


Figura 9: Grafana: throughput total consolidado

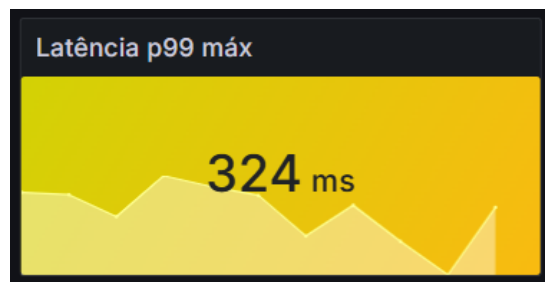


Figura 10: Grafana: latência p99 máxima registrada

3.4 Análise dos Resultados

O experimento concluiu com êxito: 1.000.000 de mensagens publicadas por 1.000 dispositivos simultâneos, sem nenhum erro, em 467,8 s. O throughput medido foi de **2.137,5 msg/s**, com latência média de 175,7 ms e p99 de 308,3 ms.

O modo de queda *session* — cada dispositivo com probabilidade de 5% de sofrer exatamente uma queda durante a sessão — gerou **50 eventos de queda** ao longo do teste, valor coerente com a probabilidade configurada aplicada sobre 1.000 dispositivos.

A ausência de erros e a estabilidade do throughput ao longo dos 467,8 s confirmam que o ThingsBoard CE, executado localmente via Docker, suporta sem degradação a carga de 1.000 dispositivos IoT publicando telemetria MQTT continuamente com intervalo de 200 ms.

4. Gráfico de Grantt

O Gráfico de Gantt abaixo demonstra a finalização da etapa de Sistema de alertas automáticos e continuando a etapa de Testes e Validações.

Sistemas Distribuídos
Grupo: José Victor, Ana Iara, Milena e Higor

Grupo: José Victor, Ana Iara, Milena e Higor

Projeto: Sistema de Monitoramento e Controle de Temperatura em Ambiente de Desenvolvimento

Resumo do Projeto

Item	Descrição	Responsável	Progresso	Prévio	Atual
1	Concepção do Projeto	José Victor / Ana Iara	100%	30/03	30/03
2	Execução	Higor / José Victor	100%	13/04	04/05
3	Monitoramento e Controle	Todos	71%	04/05	18/06
4	Encerramento	Todos	0%	08/06	22/06

Calendário de Atividades

Legenda: ■ Atividade concluída ■ Atividade em andamento ■ Atividade planejada

Calendário de Atividades (Resumo)

Atividade	Responsável	Período	Status
Concepção do Projeto	José Victor / Ana Iara	23-mar-2026 a 30/03	Concluída
Levantamento de recursos	José Victor / Ana Iara	06/04 a 13/04	Concluída
Definição da arquitetura	Higor / Milena	13/04 a 19/04	Concluída
Configuração do ambiente de dev	Todos	19/04 a 26/04	Concluída
Firmware ESP32 + leitura MPU6050	Higor / José Victor	04/05 a 11/05	Concluída
Lógica de detecção de queda	Higor / José Victor	11/05 a 18/05	Concluída
Configuração broker MQTT	Milena	18/05 a 25/05	Concluída
Integração ESP32 → ThingBoard	Ana Iara / Milena	25/05 a 01/06	Concluída
Dashboard tempo real	Milena	01/06 a 08/06	Concluída
Sistema de alertas automáticos	Milena / José	08/06 a 15/06	Concluída
Testes e Validações	Todos	15/06 a 22/06	Em andamento
Ajustes e correções	Todos	22/06 a 29/06	Em andamento